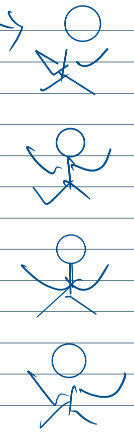
Temple



Amit
Bharma
Chirag
Deepak
$\boxed{\text{Esha} \quad \text{Farhan} \quad \text{Gita}}$

$\boxed{\text{Esha} \quad \text{Farhan} \quad \text{Gita}}$
Amit
Bharma
Chirag
Deepak

VIP quota

arr → | 1 | 2 | 3 | 4 | 5 | 6 | 7 | k = 3

op → | 5 | 6 | 7 | 1 | 2 | 3 | 4 |

Array size k = 200
||
80

	0	1	2	3	4	5	6
	1	2	3	4	5	6	7
k = 1	7	1	2	3	4	5	6
k = 2	6	7	1	2	3	4	5
k = 3	5	6	7	1	2	3	4
k = 4	4	5	6	7	1	2	3
k = 5	3	4	5	6	7	1	2
k = 6	2	3	4	5	6	7	1
k = 7	1	2	3	4	5	6	7
k = 8	7	1	2	3	4	5	6
k = 9	6	7	1	2	3	4	5
k = 10	5	6	7	1	2	3	4

10 times
||
3 times

k % n ⇒ 10 % 7 = 3

k = 200 , n = 80

k % n ⇒ 200 % 80 = 40

arr → | 1 | 2 | 3 | 4 | 5 | 6 | 7 | k = 3 $\boxed{n = 7}$

0 1 2 3 4 5 6

brr → | 5 | 6 | 7 | | | | |

brr [0] = arr [4]   n − k = 7 − 3 = 4
brr [1] = arr [5]
brr [2] = arr [6]   n − 1

brr → [5|6|7| | | |]

brr[0] = arr[6] → n - 1

j → 0 +1
j → 1 +1
j → 2

arr → [1|2|3|4|5|6|7]
0 1 2 3 4 5 6

brr → [5|6|7| | | |]
0 1 2 3 4 5 6

brr[3] = arr[0]; → 0
brr[4] = arr[1]
brr[5] = arr[2]
brr[6] = arr[3] → n - k - 1

j → 3 +1
j → 4 +1
j → 5 +1
j → 6

arr → brr
~~20K~~ 25K

Leetcode → 20K print

x = 20K
↑
print

[| | |]
1 2 3 4 5 6 7

arr	brr
~~25K~~ 25K	~~20K~~ 20K

20K [1|2|3|4|5|6|7]
25K [5|6|7|1|2|3|4]

Leetcode => x = 20K (data)

arr	brr
~~20K~~	~~25K~~
25K	20K

20K [1|2|3|4|5|6|7]
25K [5|6|7|1|2|3|4]

Leetcode => x = 20K (data) => print => 1,2,3,4,5,6,7

Leetcode ⇒ x = 20k (data) ⇒ print ⇒ 1, 2, 3, 4, 5, 6, 7

	0	1	2	3	4	5	6
arr ⇒	1	2	3	4	5	6	7

⇒ k = 3

⇓ arr1 ⇓ arr2

4 3 2 1 | 7 6 5

⇓

5 6 7 1 2 3 4

arr1 ⇒ 0 to n-k-1
arr2 ⇒ n-k to n-1

reverse(0, n-k-1)
reverse(n-k, n-1)
reverse(0, n-1)

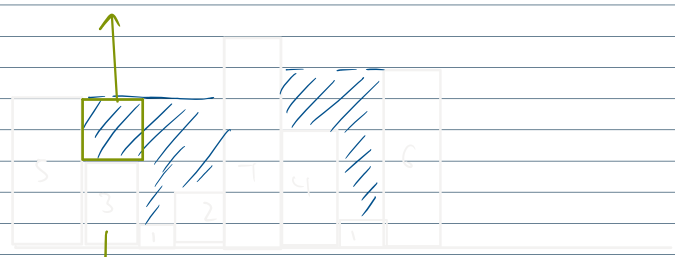
output ⇒ 5 6 7 1 2 3 4

Rain water trapping

	0	1	2	3	4	5	6	7
height ⇒	5	3	1	2	7	4	1	6

5 ⇒ 0
3 ⇒ 2 × 1 = 2
1 ⇒ 4 × 1 = 4
2 ⇒ 3 × 1 = 3 ⇒ 16
7 ⇒ 0
4 ⇒ 2 × 1 = 2
1 ⇒ 5 × 1 = 5
6 ⇒ 0 × 1 = 0

min(5, 7) - 3 = 5 - 3 = 2



(5, 7) → 5 - 3

Formula ⇒ $\boxed{\min(\text{max left}, \text{max right}) - \text{height}[i]}$

	0	1	2	3	4	5	6	7	
h →	5	3	1	2	7	4	1	6	(height array)

	0	1	2	3	4	5	6	7
left →	5	5	5	5	7	7	7	7

0	1	2	3	4	5	6	7	
7	7	7	7	7	6	6	6	← right

i = 0 ⇒ min(l[0], r[0]) - h[0] = min(5, 7) - 5 = 5 - 5 = 0

i = 1 ⇒ min(l[1], r[1]) - h[1]

i = 2 ⇒ min(l[2], r[2]) - h[2]

i = 3 ⇒ min(l[3], r[3]) - h[3]

⋮

i = 7 ⇒ min(l[7], r[7]) - h[7]

Total Sum ⇓ Answer

	0	1	2	3	4	5	6	7
h →	5	3	1	2	7	4	1	6
left →	5	5	5	5	7			

$\boxed{left[0] = h[0]}$

i = 1 ⇒ left[1] = max(l[0], h[1]) = max(3, 5) = 5

i = 2 ⇒ left[2] = max(l[1], h[2]) = max(5, 1) = 5

i = 3 ⇒ left[3] = max(l[2], h[3]) = max(5, 2) = 5

i = 4 ⇒ left[4] = max(l[3], h[4]) = max(5, 7) = 7

i = 5 ⇒ left[5] = max(l[4], h[5])

i = 6 ⇒ left[6] = max(l[5], h[6])

i = 7 ⇒ left[7] = max(l[6], h[7])

```
for(int i=1; i<n; i++) {
    left[i] = max(left[i-1], h[i]);
}
```

	0	1	2	3	4	5	6	7
h →	5	3	1	2	7	4	1	6
right →								6

$\boxed{\begin{array}{l} right[7] = h[7] \\ right[n-1] = h[n-1] \end{array}}$

n = 8 ⇒ n - 2 = 0

i = 6 ⇒ r[6] = max(r[7], h[6])

i = 5 ⇒ r[5] = max(r[6] h[5])

for(int i=n-2

r[i] = ma

i = 6 => r[6] = max(r[7], h[6])

j = 5 => r[5] = max(r[6], h[5])

j = 4 => r[4] = max(r[5], h[4])

|

|

i = 0 => r[0] = max(r[1], h[0])

```
class Solution {
    public int trap(int[] h) {
        int n = h.length;
        int[] l = new int[n];
        int[] r = new int[n];
        l[0] = h[0];
        for(int i=1; i<n; i++) {
            l[i] = Math.max(l[i-1], h[i]);
        }
        r[n-1] = h[n-1];
        for(int i=n-2; i>=0; i--) {
            r[i] = Math.max(r[i+1], h[i]);
        }
        int w = 0;
        for(int i=0; i<n; i++) {
            w = w + Math.min(l[i], r[i]) -
h[i];
        }
        return w;
    }
}
```

```
class Solution {
    public void rotate(int[] arr, int k) {
        int n = arr.length;
        k = k % n;
        reverseArrayPart(arr, 0, n-k-1);
        reverseArrayPart(arr, n-k, n-1);
        reverseArrayPart(arr, 0, n-1);
    }
    public static void reverseArrayPart(int[] arr, int i, int j) {
        while (i < j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            i++;
            j--;
```

```
            }
        }
}
```

Product of every element except self

2 3 4 3 5

$3 \times 4 \times 3 \times 5 \rightarrow 180$

$2 \times 4 \times 3 \times 5 \rightarrow 120$

$2 \times 3 \times 3 \times 5 \rightarrow 90$

$2 \times 3 \times 4 \times 5 \rightarrow 120$

$2 \times 3 \times 4 \times 3 \rightarrow 72$

op $\rightarrow$ 180 120 90 120 72

Total = 360
prod
$2 \times 3 \times 4 \times 3 \times 5$

2 3 4 3 5

$\frac{360}{2}$ $\frac{360}{3}$ $\frac{360}{4}$ $\frac{360}{3}$ $\frac{360}{5}$

180 120 90 120 72

{ Divide not allowed }

$\frac{\cancel{2} \times 3 \times 4 \times 3 \times 5}{\cancel{2}}$

$3 \times 4 \times 3 \times 5$

arr $\rightarrow$ 2 3 5 4

left $\rightarrow$ 1 2 2×3 $2 \times 3 \times 5$

1 2 6 30

arr $\rightarrow$ 2 3 5 4

right $\rightarrow$ $3 \times 5 \times 4$ 5×3 4 1

60 15 4 1

i = 0 $\Rightarrow$ left[0] × right[0] = 1 × 60 = 60 { 3× 5 × 4 }

1 × 3×5×4 = 3×5×4

i = 1 $\Rightarrow$ left[1] × right[1] = 2 × 15 = 30 { 2× 5 × 3 }

2 × 5 × 3

$\boxed{\text{product[i] = left[i] × right[i]}}$

	0	1	2	3
arr $\rightarrow$	2	3	5	4
left $\rightarrow$	1	2	6	30

	0	1	2	3
arr $\rightarrow$	2	3	5	4
right $\rightarrow$		20	4	1

left[0] = 1
left[1] = left[0] × arr[0] = 2×1 = 2
left[2] = left[1] × arr[1] = 2×3 = 6
left[3] = left[2] × arr[2] = 6×5 = 30

right[n-1] = 1
right[2] = right[3] × arr[3] = 1×4 = 4
right[1] = right[2] × arr[2] = 4×5 = 20
right[0] = right[1] × arr[1] = 20×3 = 60

```
for(int i=1;i<n;i++){
    l[i] = arr[i-1] × l[i-1];
```

```
for(int i=n-2;i>=0;i--){
    r[i] = arr[i+1] × r[i+1];
```

```
for(int i=1;i<n;i++){
    l[i] = arr[i-1] × left[i-1];
}
```

```
for(int i=n-2;i>=0;i--){
    r[i] = arr[i+1] × r[i+1];
}
```

```java
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] l = new int[n];
        int[] r = new int[n];
        l[0] = 1;
        for(int i=1; i<n; i++) {
            l[i] = l[i-1] * nums[i-1];
        }
        r[n-1] = 1;
        for(int i=n-2; i>=0; i--) {
            r[i] = r[i+1] * nums[i+1];
        }

        for(int i=0; i<n; i++) {
            nums[i] = left[i] * right[i];
        }
        return nums;
    }

}
```